Abe Busschau

Term Project: Instant Messaging App

Architecture Document

Table of Contents

1 Introduction....2

1.1 Purpose....2

1.2 Target Audience....2

1.3 Terms and Definitions....2

2 Architectural Goals and Constraints....3

2.1 Security....3

2.2 Persistency....3

2.3 Reliability/Availability....3

3 System Overview....5

4 System Architecture....9

4.1 Front-end Architecture....9

4.2 Back-End Microservices Architecture....12

5 Database Design....13

6 Issue and Concerns....14

1 Introduction

1.1 Purpose

This document describes the software architecture of the Instant Messaging App term project. It defines the system structure, architectural decisions, non-functional goals (security, reliability, scalability), and the interaction between the frontend and backend microservices.

1.2 Target Audience

This document is intended for:

- Software developers working mainly on the frontend or backend
- Instructors evaluating architectural design decisions
- Future maintainers of the system
- Team members integrating additional features

1.3 Terms and Definitions

Frontend – The user interface layer (browser or mobile client).
Backend – Server-side services handling logic and data.
Microservices – Small, independent, stateless services that implement a single responsibility
REST API – Web-based API using HTTP methods (GET, POST, PUT, DELETE).
SaaS – Software as a Service
PaaS – Platform as a Service
Stateless Service – A service that does not maintain internal state between requests
Scalability – Ability to handle increasing users
Resilience – Ability to continue functioning during failure

2 Architectural Goals and Constraints

This section describes the key architectural goals of the Instant Messaging App, focusing on non-functional system properties such as security, data persistence, and reliability. Because this is a frontend-focused project, these goals are primarily addressed through frontend design decisions and through interaction with a remote backend that provides core services.

2.1 Security

Security is an important goal for a messaging application because it handles user accounts and private conversations. From the frontend perspective, authentication is managed by the backend using a JWT stored in an HTTP-only cookie, which prevents direct access to sensitive credentials from frontend code. The frontend relies on the browser to include this cookie with authenticated requests and handles authentication failures by redirecting users. A key constraint is that authentication logic and token management are fully controlled by the backend.

2.2 Persistency

Persistence ensures that user data, contacts, and message history remain available across sessions. In this project, all persistent data is managed by the backend, while the frontend retrieves and updates data through RESTful API calls. Messages delivered in real time through WebSocket connections are also persisted by the backend and can later be retrieved using standard API requests. A constraint of this approach is that the frontend has no direct control over database structure or storage mechanisms.

2.3 Reliability/Availability

Reliability and availability are essential for providing a usable chat experience. The frontend addresses these goals by handling slow or failed backend responses gracefully through loading indicators, error messages, and connection status feedback. For real-time

messaging, the frontend monitors WebSocket connection state and responds to disconnects when necessary. Overall system availability depends on the external backend and network conditions, which limit frontend control and require a resilient user interface design.

3 System Overview

This project is a web-based Instant Messaging Application that allows users to authenticate, join chat rooms, and exchange messages in real time through a browser-based client. The system follows a client–server architecture, where the frontend (client) is responsible for user interaction and interface management, while the backend (server) provides application services such as authentication, chat room management, and message delivery.

At a high level, the system is organized using a layered architectural model, where each layer represents a coherent area of concern.

High-level system functionality and components

User Interface (UI)

The **User Interface layer** is the part of the system that users directly interact with through their web browser. It provides visual components and screens for:

- User login and registration
- Viewing the available chat room
- Navigating between chat rooms
- Viewing message history within a room
- Sending messages
- Managing basic user profile information

This layer focuses purely on presentation and interaction, without containing business logic or direct data access.

User Interface Management

The **UI Management layer** controls how the user interface behaves and how different screens are connected. It includes:

- Client-side routing and navigation between views (e.g., login screen, room list, chat room)
- Session handling (tracking whether a user is logged in)
- Frontend state management (current user, active room, message list)
- Communication logic for sending requests to the backend and handling responses
- Client-side validation and error handling to improve user experience

By separating UI management from visual components, the frontend is decomposed into smaller, self-contained parts, which reduces complexity and improves maintainability (Chapter 4).

Application Services

The **Application Services layer** contains the core functionality of the system and is implemented on the backend. In this project, the backend is provided externally and accessed only through documented APIs. At a conceptual level, application services include:

- User Account Services (authentication, login, profile management)
- Chat Room Management Services (creating, listing, and deleting chat rooms)
- Message Management Services (sending messages, retrieving chat history, and real-time message delivery)

These services represent coherent sets of features, rather than individual user actions.

Shared Infrastructure Services

The **Shared Infrastructure Services layer** provides cross-cutting support used by multiple application services. These services are not directly visible to users but are essential for system operation. They include:

- A single backend entry point (API gateway or base URL) used by the frontend
- Security mechanisms such as authentication checks
- Logging, monitoring, and error reporting

These shared services support reliability, security, and availability across the system.

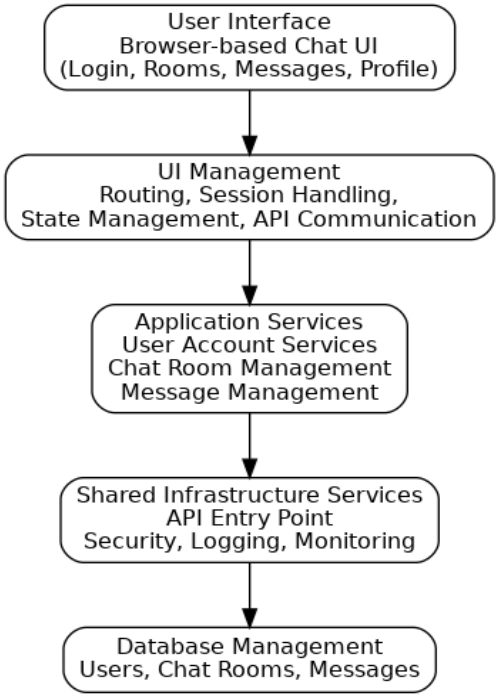
Database Management

The **Database Management layer** is responsible for persistent data storage, including:

- User account information
- Chat room data
- Message history

In this frontend-focused project, the database is fully managed by the backend. The frontend does not access the database directly; instead, it achieves data persistence and retrieval by communicating with backend application services through APIs. This separation ensures loose coupling between the user interface and data storage.

Layered Architectural Model Diagram:


User Interface
Browser-based Chat UI
(Login, Rooms, Messages, Profile)
UI Management
Routing, Session Handling,
State Management, API Communication
Application Services
User Account Services
Chat Room Management
Message Management
Shared Infrastructure Services
API Entry Point
Security, Logging, Monitoring
Database Management
Users, Chat Rooms, Messages

4 System Architecture

4.1 Front-end Architecture

Provide a one-page, detailed description of the frontend architecture of your system. It should include the frontend framework or library, presentation, routing, functionalities, API components, and any cloud-based software or platform (Chapter 5) you have used. Specify if you have used any PaaS or SaaS (Chapter 5 p16).
Then summarize your description into an architecture diagram.

The Instant Messaging App frontend is a browser-based Single Page Application (SPA) built using Vite. As the frontend developer for this project, my responsibility is the client-side architecture, which includes presentation, routing and navigation, session and state management, and communication with the remote backend. The backend is provided externally and is treated as a black-box service, accessed through documented APIs. The frontend uses RESTful HTTP communication (HTTP verbs) for all standard operations and a WebSocket-based channel for real-time messaging.

Presentation and component decomposition

The frontend user interface is decomposed into small, self-contained components, where each component represents a coherent set of functionality rather than a single monolithic UI. This approach minimizes complexity and improves maintainability by limiting how changes in one part of the interface affect others.

At a high level, the presentation layer is organized into the following coherent UI areas:

- **Authentication and Profile UI**: user signup, login, logout, and profile updates.
- **Contacts UI**: searching users, listing contacts, and selecting a direct message thread.

- **Messaging UI**: displaying message history and composing new messages with real-time updates.

Routing and navigation

Client-side routing is used to manage navigation between major views and provide smooth transitions without reloading the page. Typical routes include authentication views, a contacts list view, and a direct messaging view. Routing is also used to enforce basic access control on the client side, redirecting unauthenticated users to the login view when necessary.

Frontend state and session management

The frontend maintains the minimal application state required for a messaging system, including the current authenticated user, the active contact or direct message thread, message history for the selected thread, and UI state such as loading or error indicators. Authentication is handled using a JWT stored in an HTTP-only cookie set by the backend. The frontend does not read this cookie directly. It rather relies on the browser automatically including it with requests, requiring API calls to be made with credentials enabled.

Communication architecture

The frontend uses two communication mechanisms, based on the backend API specification:

RESTful HTTP (JSON) is used for all non-real-time operations, including user authentication, profile updates, retrieving contact lists, creating or deleting direct message threads, and fetching message history. These operations use appropriate HTTP verbs such as GET, POST, and DELETE. To ensure correct authentication and connectivity, the frontend configures requests with (withCredentials: true) and includes the required ngrok bypass header.

WebSocket-based communication (Socket.IO) is used exclusively for real-time messaging. The frontend emits events to send messages and listens for events to receive incoming messages, enabling low-latency, bidirectional communication.

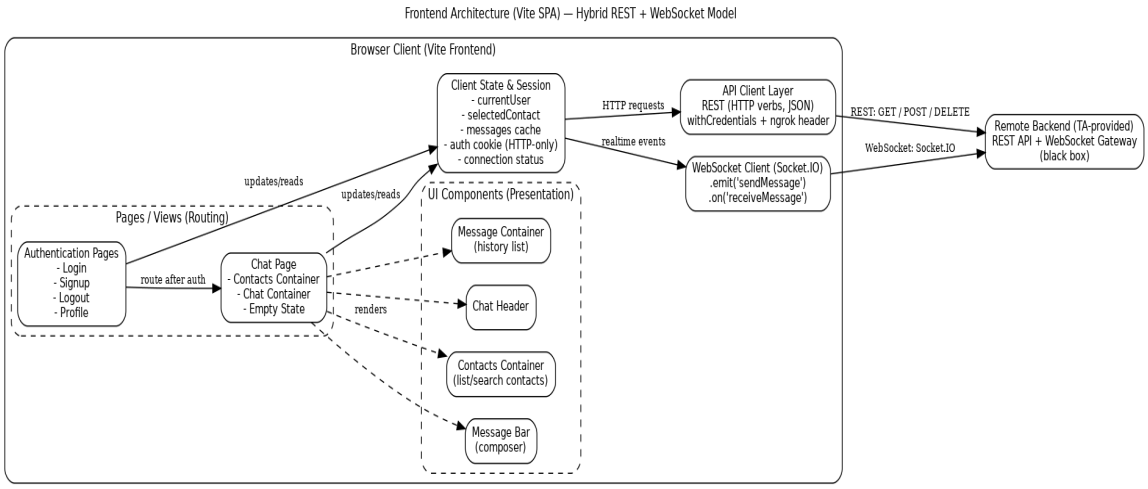
Error handling and user experience

Because the backend is remote and network conditions may vary, the frontend includes explicit error-handling behavior. This includes loading states during API calls, clearing user-facing error messages for authentication or validation failures, and handling WebSocket disconnections with reconnection indicators. This improves usability while maintaining a clean separation of concerns in the frontend architecture.

Cloud-based delivery and source management

The frontend is delivered to users as a web-based SaaS client, accessed through a standard web browser. The frontend source code is maintained and submitted through a GitHub repository for version control and project submission. GitHub is used for source hosting rather than as a runtime execution platform; the built frontend artifacts produced by Vite can be served by any appropriate static hosting environment.

Diagram:



4.2 Back-end Microservices Architecture

The backend of this application is provided externally and is not implemented as part of the frontend-focused project. From the frontend's perspective, the backend is treated as a black-box service that exposes documented REST APIs for authentication, contact management, and message history, as well as a WebSocket gateway for real-time messaging.

At a high level, the backend logically supports services such as User Account Management, Contact/Conversation Management, and Message Delivery, which may be implemented following a microservices architecture. The frontend does not depend on backend implementation details and interacts with these services only through defined interfaces.

5 Database Design

The Instant Messaging App relies on persistent data storage to support user accounts, contact lists, and message history. In this frontend-focused project, the database is fully managed by the backend, which is provided externally. MangoDB is used for this case. The frontend does not directly access or manage the database. It rather interacts with persistent data exclusively through documented backend APIs.

At a high level, the system's persistent data includes:

- **User data** (account credentials and profile information)
- **Contact and conversation data** (direct message relationships)
- **Message data** (message content, sender, timestamps)

The frontend achieves data persistence and retrieval by making **RESTful HTTP requests** to backend endpoints for operations such as authentication, retrieving contacts, and fetching message history. These requests use standard HTTP verbs (GET, POST, DELETE) and exchange data in JSON format. Authentication information is maintained using a JWT stored in an HTTP-only cookie, which is automatically included with requests by the browser.

For real-time messaging, the frontend uses a **WebSocket-based communication channel** to send and receive messages instantly. Messages delivered through the WebSocket are persisted by the backend and can later be retrieved through REST API calls, ensuring consistency between live communication and stored message history.

This design achieves interoperability between the user interface and background data by clearly separating responsibilities: the frontend focuses on presentation and user interaction, while the backend handles data storage, consistency, and persistence. This separation reduces coupling and allows the frontend to remain independent of database implementation details.

6 Issue and Concerns

While developing the frontend for the Instant Messaging App, a few practical challenges came up that influenced architectural decisions. Since the backend is provided externally and treated as a black box, the frontend had to be designed to handle delayed responses and errors gracefully, rather than assuming everything would always work perfectly.

Another challenge was working with two different communication styles in the same application. Most features use standard HTTP requests, while live messaging relies on a WebSocket connection. Keeping these two paths clearly separated in the frontend will help prevent confusing state updates and make the code easier to reason about.

Managing frontend state was also a consideration, especially because chat applications are highly dynamic. Changes in routes, authentication status, incoming messages, and connection state can all happen at once. Separating state management from presentation components made the UI more predictable and easier to maintain.

Finally, using authentication based on HTTP-only cookies required attention when configuring requests, since the frontend cannot directly access the authentication token. This will affect how API calls and WebSocket connections are initialized and tested.